생산라인 모니터링 시스템

DB 및 데이터 통신 규격서

Raspberry Pi · OpenCV · TCP/IP · C# WinForms · MFC · SQLite

문서 목적	팀원별 기능 결과를 중앙 서버 및 SQLite DB 에 안정적으로 연동하기 위한 공통 규격 정의
DB 관리자	김초아
대상 인원	김초아 · 조시현 · 노종헌 · 노원종 · 유동호
DB 파일	production.db
작성일	2026-10-13

핵심 원칙

각 팀원은 DB 를 직접 수정하지 않고, 정해진 데이터 규격에 맞춰 서버로 전달합니다.
DB 저장·조회·통계 연동은 메인 서버에서 일괄 처리합니다.

1. 문서 개요

본 문서는 생산라인 타이어 검사·설비 제어 프로젝트에서 발생하는 검사 결과, 설비 상태, 채팅, 인수인계 데이터를 하나의 기준으로 통합하기 위한 규격서입니다. **SQLite DB 는 김초아가 단독 관리**하며, 다른 팀원은 SQL 이나 DB 파일을 직접 수정하지 않고 본 문서의 필드 규격에 맞춰 데이터를 전달합니다.

1.1 적용 범위

- Raspberry Pi 및 OpenCV 검사 결과 데이터
- 컨베이어·센서·서보·LED·부저 등 설비 상태 및 이벤트 데이터
- C# WinForms / MFC 실시간 채팅 데이터
- 작업자 인수인계 데이터
- MFC 생산 대시보드·검사 이력·그래프·일일 보고서용 DB 조회 데이터

1.2 기본 운영 원칙

원칙	내용
DB 단일 관리	production.db 는 메인 서버 PC 에서 관리하고, DB 구조 변경은 김초아가 담당합니다.
직접 접근 금지	Raspberry Pi, WinForms, MFC 등 각 클라이언트가 production.db 를 직접 열어 동시에 수정하지 않습니다.
데이터 규격 고정	각 팀원은 지정된 type 및 필드명과 값 규칙을 사용합니다.
서버 기준 저장	시간 기록과 DB INSERT/UPDATE 는 메인 서버에서 처리합니다.
NG 이미지 경로 저장	NG 이미지는 파일로 보관하고 DB 에는 이미지 자체가 아닌 경로를 저장합니다.

2. 전체 시스템 및 데이터 흐름

메인 서버를 중심으로 각 프로그램의 데이터를 수집하고, DB 저장 후 필요한 화면에 다시 제공하는 구조를 사용합니다.

OpenCV / Hardware	TCP/IP	Central Server	SQLite	MFC / WinForms
검사·설비 데이터	전송	분류·저장·중계	production.db	조회·표시·제어

OpenCV / 설비 → TCP/IP → 중앙 서버 → SQLite 저장 → MFC·WinForms 활용

3. 사용자 및 공통 이름 규칙

users 테이블에는 아래 5 명만 등록하며, user_name 값은 아래 한글 이름을 그대로 사용합니다.

ID	user_name	비고
1	김초아	등록 사용자
2	조시현	등록 사용자
3	노종헌	등록 사용자
4	노원종	등록 사용자
5	유동호	등록 사용자

주의: choa, 김 초아, kimchoa 등 임의 표기를 사용하지 않고 반드시 등록된 user_name 문자열을 사용합니다.

4. DB 테이블 구성

테이블	용도	주요 컬럼
users	작업자 정보	id, user_name
inspection	타이어 검사 결과	id, tire_id, result, defect_type, image_path, detected_at
equipment_log	설비 상태 및 이벤트	id, equipment, status, message, created_at
chat	실시간 채팅 기록	id, user_name, message, created_at
handover	작업자 인수인계	id, user_name, message, checked, created_at

5. 공통 데이터 규격

모든 전송 데이터는 type 값으로 종류를 구분합니다. 필드명은 아래 규격을 그대로 사용하고, 가능한 한 임의의 별칭을 만들지 않습니다.

5.1 INSPECTION - 타이어 검사 결과

필드	예시	설명
tire_id	TIRE-001	타이어 고유 번호
result	OK / NG	검사 결과. 반드시 OK 또는 NG 사용
defect_type	CRACK	불량 종류. 정상인 경우 null 또는 빈 값
image_path	NG_Images/TIRE-001.jpg	NG 이미지 저장 경로. 없으면 null 또는 빈 값

※ detected_at 은 클라이언트가 보내지 않고 서버에서 DB 저장 시 자동 기록합니다.

5.2 EQUIPMENT - 설비 상태 및 이벤트

필드	예시	설명
equipment	CONVEYOR	설비 식별값
status	RUN / STOP / ERROR	현재 상태
message	컨베이어 가동 시작	상태 또는 이벤트 설명

※ created_at 은 서버 저장 시 자동 기록합니다.

5.3 CHAT - 실시간 채팅

필드	예시	설명
user_name	김초아	등록 사용자 이름
message	라인 확인해주세요.	채팅 내용

※ created_at 은 서버 저장 시 자동 기록합니다.

5.4 HANDOVER - 작업자 인수인계

필드	예시	설명
user_name	조시현	등록 사용자 이름
message	카메라 위치 확인 필요	인수인계 내용

※ checked 값은 최초 저장 시 서버가 0(미확인)으로 설정하고, 확인 처리 시 1 로 변경합니다.

6. 값 표준화 규칙

항목	허용 값	규칙
검사 결과 result	OK, NG	다른 표현(NORMAL, PASS, FAIL 등) 사용 금지
설비 equipment	CONVEYOR, CAMERA, SENSOR, SERVO, LED, BUZZER, RASPBERRY_PI	영문 대문자 기준
설비 status	RUN, STOP, ON, OFF, ERROR, CONNECTED, DISCONNECTED	상태 표현을 고정
사용자 user_name	김초아, 조시현, 노종헌, 노원종, 유동호	users 테이블 등록명 그대로 사용
시간	서버 자동 생성	클라이언트 전송 불필요

7. TCP/IP 전송 예시(JSON)

TCP/IP 담당자는 아래와 같은 JSON 구조를 기준으로 메시지를 구성하면 서버에서 type 에 따라 분기하기 쉽습니다.

INSPECTION

```
{
  "type": "INSPECTION",
  "tire_id": "TIRE-001",
  "result": "NG",
  "defect_type": "CRACK",
  "image_path": "NG_Images/TIRE-001.jpg"
}
```

EQUIPMENT

```
{
  "type": "EQUIPMENT",
  "equipment": "CONVEYOR",
  "status": "RUN",
  "message": "컨베이어 가동 시작"
}
```

CHAT

```
{
  "type": "CHAT",
  "user_name": "김초아",
  "message": "생산라인 확인해주세요."
}
```

HANDOVER

```
{
  "type": "HANDOVER",
  "user_name": "조시현",
  "message": "카메라 위치 재확인 필요"
}
```

8. 담당자별 연동 기준

기능	담당	전송 구분	연동 기준
OpenCV 검사 담당	노종헌 · 조시현	INSPECTION	tire_id, result, defect_type, image_path 를 규격에 맞게 생성
설비/자동화 담당	김초아 · 조시현 · 유동호 · 노원종	EQUIPMENT	설비 상태 변화 및 오류 발생 시 equipment/status/message 생성
TCP/IP 담당	유동호 · 노원종 · 노종헌	전체 type	메시지 전송·수신 및 type 별 분기. DB 직접 수정 불필요
채팅/인수인계	유동호 · 노원종	CHAT / HANDOVER	등록 user_name 과 message 전달
DB/통합 관리	김초아	SQLite / MFC	DB 저장·조회, 대시보드, 그래프, 보고서 및 최종 연동

9. 연동 절차

단계	작업	확인 내용
1	팀원 기능 단독 테스트	각 파트에서 필요한 데이터 값이 정상적으로 생성되는지 확인
2	규격 확인	type, 필드명, 허용 값이 본 문서와 동일한지 확인
3	TCP/IP 전송 테스트	서버에 JSON 메시지가 정상 도착하는지 확인
4	서버 분기 테스트	type 별로 INSPECTION / EQUIPMENT / CHAT / HANDOVER 처리가 분리되는지 확인
5	SQLite 저장 테스트	각 메시지가 지정된 테이블에 정상 저장되는지 확인
6	MFC/WinForms 조회 테스트	저장된 데이터가 대시보드·이력·로그·채팅 화면에 표시되는지 확인
7	통합 테스트	OpenCV → TCP/IP → 서버 → DB → MFC 전체 흐름 검증

10. 최종 체크리스트

- ☐ production.db 는 메인 서버 PC 에서만 관리한다.
- ☐ 팀원은 DB 파일을 직접 열어 INSERT/UPDATE 하지 않는다.
- ☐ 전송 type 은 INSPECTION / EQUIPMENT / CHAT / HANDOVER 중 하나를 사용한다.
- ☐ result 는 OK 또는 NG 로 통일한다.
- ☐ user_name 은 등록된 5 명 이름만 사용한다.
- ☐ 시간 데이터는 서버에서 자동 기록한다.
- ☐ NG 이미지는 파일로 저장하고 DB 에는 image_path 를 기록한다.
- ☐ TCP/IP 연동 전 각 파트가 생성하는 데이터 값부터 단독 확인한다.
- ☐ 최종 통합 시 OpenCV → 서버 → DB → MFC 흐름을 순서대로 검증한다.

11. 최종 요약

팀원은 각자 담당 기능에서 규격화된 데이터만 생성·전송하고, 김초아가 메인 서버와 SQLite DB 를 중심으로 데이터를 통합 관리합니다. **DB 직접 접근을 한 곳으로 제한**함으로써 구조 변경, 중복 저장, 필드명 불일치와 같은 통합 오류를 줄이고, 이후 MFC 대시보드·그래프·보고서 기능까지 동일한 데이터 기준으로 연결할 수 있습니다.

팀원 전달 문구

"DB 는 제가 관리할 테니, 각자 기능에서 위 규격대로 데이터만 넘겨주세요.
SQL 이나 production.db 파일은 직접 수정하지 않아도 됩니다."